



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

RELAZIONE DEL PROGETTO:

"Applicazione del Reinforcement Learning e dei Large Language Model al gioco Sokoban"

A CURA DI:

Antonio Di Giorgio

Matricola: 0522502003

Codice sorgente: <https://github.com/AntonioDG30/IA-Sokoban>



Anno Accademico 2025-2026

Indice

1	Introduzione	1
1.1	Contesto e motivazioni	1
1.2	Obiettivi del progetto	2
1.3	Contributi principali	2
1.4	Struttura della relazione	3
2	Fondamenti Teorici	5
2.1	Reinforcement Learning	5
2.1.1	Il problema decisionale sequenziale	5
2.1.2	Il modello formale: MDP	5
2.1.3	Policy e funzioni di valore	6
2.2	Algoritmi di RL profondo	7
2.2.1	Q-Learning e Deep Q-Network	7
2.2.2	Proximal Policy Optimization	7
2.3	Architetture neurali	8
2.3.1	Reti convoluzionali	8
2.3.2	Memoria ricorrente: LSTM	9
2.4	Large Language Model	9
2.4.1	Architettura e capacità	9
2.4.2	LLM come agenti decisionali	10

2.5	Curriculum Learning	10
2.6	Learning from Demonstrations	11
3	Stato dell'Arte	12
3.1	RL applicato ai videogiochi	12
3.2	Sokoban come benchmark per il RL	13
3.3	Curriculum Learning nei task combinatori	13
3.4	LLM come agenti di pianificazione	14
3.5	Integrazione LLM e Reinforcement Learning	14
4	Il gioco Sokoban: ambiente e dataset	15
4.1	Regole e meccaniche di gioco	15
4.2	Complessità computazionale	16
4.3	Dataset DeepMind Boxoban	16
4.4	Ambiente custom con Gymnasium (sistema 10×10)	16
4.4.1	Rappresentazione dello stato	17
4.4.2	Spazio delle azioni	17
4.4.3	Sistema di ricompense	17
4.5	Sistema semplificato 7×7	18
4.5.1	Motivazioni	18
4.5.2	Generatore di livelli e verifica di risolvibilità	19
5	Metodologia	21
5.1	Approccio generale al problema	21
5.2	Curriculum learning: struttura e fasi	22
5.2.1	Sistema 10×10 : curriculum adattivo a sei fasi	22
5.2.2	Sistema 7×7 : curriculum semplificato	22
5.2.3	Curriculum adattivo e gestione del budget	23
5.3	Architettura della rete neurale convoluzionale	23
5.4	Design dei cinque agenti	24
5.4.1	AG-PPO: baseline con memoria ricorrente	24
5.4.2	AG-DQN: apprendimento off-policy	25
5.4.3	AG-LLM-ACT: LLM come controllore diretto	25

5.4.4	AG-LLM-GUIDE: learning from demonstrations	26
5.4.5	AG-LLM-REW: LLM come reward model	26
5.5	Integrazione del Large Language Model	27
5.5.1	Scelta del modello: da llama3.1 a qwen3:14b	27
5.5.2	Progettazione del prompt	27
5.5.3	Gestione della latenza	29
6	Implementazione	30
6.1	Stack tecnologico	30
6.2	Ambiente di simulazione	31
6.2.1	Il sistema 10×10	31
6.2.2	Il sistema 7×7	32
6.2.3	Test unitari	32
6.3	CNN e adattamento all'interfaccia SB3	33
6.4	Training di AG-PPO	33
6.5	Training di AG-DQN	34
6.6	Valutazione di AG-LLM-ACT	34
6.7	Training di AG-LLM-GUIDE	35
6.8	Training di AG-LLM-REW	35
6.9	Client LLM	36
6.10	Problemi incontrati e soluzioni adottate	37
7	Esperimenti e Risultati	39
7.1	Setup sperimentale	39
7.2	Sistema 10×10 : risultati per fase	40
7.2.1	Fase C0: 1 cassa, livelli generati	40
7.2.2	Fasi C1–C5: solve rate a zero	40
7.3	Sistema 7×7 : risultati per fase	41
7.3.1	Fase C0: 1 cassa, livelli generati 7×7	41
7.3.2	Fasi C1 e C2 con rispettivamente 2 e 3 casse	42
7.4	Analisi comparativa	42
7.5	Discussione	43

8 Conclusioni	45
8.1 Riepilogo dei contributi	45
8.2 Limiti	45
8.3 Lettura dei risultati	46
8.4 Sviluppi possibili	46
Bibliografia	48

CAPITOLO 1

Introduzione

1.1 Contesto e motivazioni

Sokoban fu ideato dal programmatore giapponese Hiroyuki Imabayashi nel 1982. Le regole sono semplici: il giocatore spinge casse su una griglia bidimensionale fino a portarle sui bersagli. La difficoltà è tutt'altra cosa, infatti, Sokoban è PSPACE-completa [1], una classe più alta di NP: non esistono algoritmi in grado di risolvere istanze arbitrarie del problema in tempo polinomiale.

La proprietà che lo rende ostile ai metodi di ricerca classici è l'irreversibilità delle spinte. Una cassa finita in un angolo tra due muri non si può recuperare. Un agente deve quindi pianificare su decine o centinaia di mosse, e l'esplorazione casuale non genera abbastanza episodi di successo da avviare l'apprendimento.

Il Reinforcement Learning (RL) e i Large Language Model (LLM) sono stati applicati a problemi con queste caratteristiche. Il primo impara per interazione diretta con l'ambiente, senza che gli venga detto quale mossa è corretta. I secondi, addestrati su grandi corpus testuali, mostrano capacità di ragionamento su descrizioni simboliche di stati e azioni, inclusa la pianificazione sequenziale. Questo progetto applica entrambi a Sokoban e ne confronta i risultati.

1.2 Obiettivi del progetto

Il progetto è stato sviluppato per il corso di Intelligenza Artificiale dell'Università degli Studi di Salerno. La traccia richiede tre agenti: uno basato su RL puro, uno in cui un LLM decide l'azione ad ogni turno in base allo stato corrente, e uno in cui l'LLM opera come reward model, valutando le mosse e assegnando ricompense aggiuntive. Il confronto tra questi tre approcci, misurato su solve rate e numero di mosse, è l'obiettivo centrale della valutazione.

© Obiettivo della Ricerca

Sviluppare e confrontare agenti intelligenti basati su Reinforcement Learning e Large Language Model applicati al gioco Sokoban, valutando l'efficacia di ciascun approccio e delle loro integrazioni nel risolvere livelli di complessità crescente.

Il lavoro include due agenti oltre ai requisiti minimi. AG-DQN è un secondo agente RL con architettura DQN, aggiunto per separare l'effetto delle scelte algoritmiche dall'integrazione con l'LLM. AG-LLM-GUIDE integra l'LLM nel training secondo il paradigma del Learning from Demonstrations: il modello linguistico gioca autonomamente su ciascuna fase del curriculum, le transizioni vengono salvate, e il DQN viene poi addestrato con il replay buffer pre-riempito con queste dimostrazioni.

1.3 Contributi principali

Il progetto ha prodotto cinque agenti, un ambiente di simulazione e gli script di training e valutazione comparativa. L'ambiente è in Python, compatibile con Gymnasium [2] e dotato di renderer grafico basato su Pygame. Ha due configurazioni: griglia 10×10 con livelli dal dataset DeepMind Boxoban [3], e griglia 7×7 nativa con livelli generati proceduralmente e verificati risolvibili tramite BFS. I cinque agenti implementati sono:

- **AG-PPO**: PPO con memoria ricorrente LSTM, agente baseline principale;
- **AG-DQN**: DQN con replay buffer, confronto off-policy;

- **AG-LLM-ACT**: LLM come controllore diretto ad ogni step, senza training RL;
- **AG-LLM-GUIDE**: DQN addestrato su dimostrazioni raccolte dall'LLM;
- **AG-LLM-REW**: PPO con reward aggiuntiva fornita dall'LLM ad ogni spinta di cassa.

Il training usa un curriculum adattivo a più fasi: si parte da una cassa su livelli generati e si arriva a quattro casse su livelli Boxoban (nel caso della configurazione con griglia 10×10). L'LLM è qwen3:14b-q4_K_M, eseguito localmente via Ollama con un client HTTP a connessione persistente per contenere la latenza.

La valutazione copre 100 episodi per fase per ciascun agente. La metrica principale è il solve rate sugli episodi terminati con successo, affiancato da reward cumulativa media, mosse medie negli episodi risolti e numero medio di casse posizionate correttamente.

1.4 Struttura della relazione

La relazione è suddivisa nei seguenti capitoli:

- Il **Capitolo 2** copre i fondamenti teorici: MDP, policy, value function, algoritmi PPO e DQN, Large Language Model, Curriculum Learning e Learning from Demonstrations.
- Il **Capitolo 3** contestualizza il lavoro rispetto alla letteratura su RL applicato ai giochi, Sokoban come benchmark, LLM per la pianificazione e integrazione RL-LLM.
- Il **Capitolo 4** descrive il dominio: regole di Sokoban, complessità computazionale, dataset Boxoban e le due varianti dell'ambiente custom.
- Il **Capitolo 5** dettaglia le scelte metodologiche: struttura del curriculum, architettura CNN e design dei cinque agenti.
- Il **Capitolo 6** tratta l'implementazione di ciascun componente, dallo stack tecnologico al client LLM.

- Il **Capitolo 7** presenta i risultati con analisi comparativa sui due sistemi (10×10 e 7×7).
- Il **Capitolo 8** chiude con un bilancio dei contributi, i limiti del lavoro e le possibili direzioni future.

2.1 Reinforcement Learning

2.1.1 Il problema decisionale sequenziale

Nel Reinforcement Learning (RL) un agente impara a prendere decisioni interagendo direttamente con un ambiente, senza che nessuno gli mostri la risposta corretta. L'unica informazione disponibile è un segnale numerico di reward che arriva dopo ogni azione: un valore positivo se la mossa è stata utile, negativo se ha peggiorato la situazione. Partendo da questo solo feedback, l'agente deve scoprire da solo quale comportamento massimizza la reward raccolta nel tempo.

Il ciclo base si ripete a ogni passo: l'agente osserva lo stato corrente dell'ambiente, sceglie un'azione, riceve la reward associata e passa allo stato successivo. Il processo continua fino al termine dell'episodio o al raggiungimento del limite di mosse.

2.1.2 Il modello formale: MDP

Il framework formale che descrive questo tipo di problema è il Processo di Decisione di Markov (MDP) [4]. Un MDP definisce quattro elementi: lo spazio degli stati possibili (tutte le configurazioni dell'ambiente), lo spazio delle azioni disponibili,

la funzione che descrive come le azioni cambiano lo stato, e la reward associata a ogni transizione. A questi si aggiunge il fattore di sconto γ , che dà meno peso alle reward lontane nel tempo rispetto a quelle immediate. L'agente punta a massimizzare la somma scontata di tutte le reward future dall'istante corrente in poi.

La proprietà di Markov richiede che la distribuzione sullo stato successivo dipenda solo dallo stato e dall'azione attuali, non dall'intera storia dell'episodio. Quando questa proprietà non vale naturalmente, si può soddisfarla arricchendo la rappresentazione dello stato. In questo progetto AG-PPO usa una memoria ricorrente LSTM esattamente per questo motivo: la griglia istantanea non contiene informazioni sulle mosse passate, e la LSTM le accumula nel proprio stato interno.

2.1.3 Policy e funzioni di valore

Una *policy* è la regola di comportamento dell'agente: data la situazione corrente, quale azione scegliere. Può essere deterministica (un'azione sola per ogni stato) oppure stocastica (una distribuzione di probabilità sulle azioni).

Per ragionare su quanto valga una certa situazione o una certa scelta, il RL usa due funzioni. La *value function* misura la reward totale attesa a partire da uno stato, seguendo una determinata policy fino al termine dell'episodio. La *Q-function* misura la stessa quantità ma condizionando anche sull'azione: quanto vale compiere questa specifica mossa in questa specifica situazione, e poi seguire la policy? Entrambe le funzioni dipendono dalla policy in uso: la stessa griglia di Sokoban vale poco per un agente che non sa pianificare e molto per un agente che conosce la sequenza di spinte giusta.

Il training RL cerca la policy che massimizza il valore atteso: quelle della *policy ottimale* e della *Q-function ottimale* sono legate dalle equazioni di Bellman, relazioni ricorsive che esprimono il valore di uno stato come la reward immediata più il valore scontato dello stato successivo. Queste equazioni sono alla base di tutti gli algoritmi che seguono.

2.2 Algoritmi di RL profondo

2.2.1 Q-Learning e Deep Q-Network

Il Q-Learning [5] aggiorna la stima del valore di ogni coppia (stato, azione) usando l'esperienza reale. Dopo aver eseguito una mossa e osservato la reward ottenuta, l'agente corregge la propria stima di Q avvicinandola al valore reale: la reward appena ricevuta più il valore stimato della situazione in cui si trova ora. Con spazi di stato piccoli funziona bene; con stati ad alta dimensione, come una griglia di gioco, la tabella esplicita di tutti i valori Q diventa impraticabile.

Il Deep Q-Network (DQN), proposto da Mnih et al. [6], sostituisce la tabella con una rete neurale che approssima la Q-function. Il problema è che addestrare una rete neurale sull'esperienza raccolta in sequenza tende a essere instabile: le esperienze consecutive sono correlate tra loro, e il target che si vuole raggiungere cambia a ogni aggiornamento della rete stessa. DQN risolve entrambi i problemi con due accorgimenti. Il primo è l'*Experience Replay*: le transizioni osservate vengono accumulate in un buffer di dimensione fissa e campionate a caso durante il training, invece di usarle nell'ordine in cui sono arrivate. Campionare a caso rompe la correlazione tra esperienze temporalmente vicine. Il secondo è la *Target Network*: una seconda copia della rete, aggiornata meno frequentemente, calcola i target invece della rete principale. I target restano stabili per molti aggiornamenti di fila, il che stabilizza l'ottimizzazione.

In questo progetto AG-DQN usa un replay buffer da 100.000 transizioni che viene mantenuto tra una fase del curriculum e l'altra: le esperienze delle fasi precedenti continuano a partecipare al training anche dopo il cambio di distribuzione dei livelli.

2.2.2 Proximal Policy Optimization

PPO [7] aggiorna direttamente la policy invece di passare per la Q-function. Il metodo precedente della stessa famiglia, TRPO, imponeva un vincolo esplicito che impediva aggiornamenti troppo grandi, ma richiedeva calcoli computazionalmente costosi. PPO ottiene lo stesso risultato con un meccanismo più semplice: misura quanto la policy nuova differisca da quella vecchia tramite il rapporto tra le probabilità

delle azioni, poi taglia gli aggiornamenti che spingerebbero questo rapporto fuori da un intervallo prefissato. Il parametro ϵ controlla l'ampiezza dell'intervallo; in questo progetto vale 0.2, il valore di default di SB3.

PPO è un algoritmo on-policy: usa solo l'esperienza raccolta dalla policy corrente per aggiornarla, poi scarta quella esperienza. Questo lo rende più stabile di DQN su task con reward sparsa, ma meno efficiente nel riutilizzo dei dati.

In questo progetto AG-PPO usa `RecurrentPPO` da `sb3-contrib` [8], una variante che integra una cella LSTM nella rete di policy e value. La ragione è pratica: la griglia osservata istante per istante non dice all'agente dove è già stato né quali configurazioni ha già raggiunto. La LSTM può accumularle nel proprio stato interno durante l'episodio.

2.3 Architetture neurali

2.3.1 Reti convoluzionali

Le reti convoluzionali (CNN) sono progettate per dati con struttura spaziale. Invece di collegare ogni neurone a ogni pixel dell'input, una CNN applica filtri locali che analizzano piccole porzioni dell'immagine e scorrono su tutta la mappa. Il vantaggio è che gli stessi filtri riconoscono la stessa struttura indipendentemente da dove si trovi: un angolo formato da due muri vicini al bordo superiore e uno al centro attivano lo stesso filtro. Le reti fully-connected non hanno questa proprietà e devono imparare ogni configurazione separatamente per ogni posizione.

Lo stato di gioco è una matrice di interi dove ogni cella indica il tipo di elemento presente (muro, pavimento, target, cassa, giocatore, padding). La `SokobanCNN` ha tre strati convoluzionali con 32 e 64 filtri e uno strato lineare finale che produce un vettore di 256 numeri. Questo vettore è l'input all'algoritmo RL: rappresenta la situazione di gioco in uno spazio compatto su cui la rete di policy può ragionare.

La stessa architettura funziona sia sul sistema 10×10 che su quello 7×7 : la dimensione del layer di flatten dipende dalla dimensione dell'input (1600 per griglie 10×10 , 1024 per griglie 7×7) e viene calcolata automaticamente con una forward pass su un tensore dummy alla costruzione del modello, invece di essere hardcoded.

L'output è sempre un vettore di 256 numeri indipendentemente dalla dimensione della griglia.

2.3.2 Memoria ricorrente: LSTM

Le Long Short-Term Memory (LSTM), introdotte da Hochreiter e Schmidhuber [9], sono reti ricorrenti che mantengono uno stato interno tra un passo e il successivo. Il problema delle reti ricorrenti semplici è che durante il training il segnale di errore si attenua o si amplifica scorrendo a ritroso nel tempo attraverso molti passi, rendendo impossibile imparare dipendenze a lungo raggio. La LSTM risolve questo con tre meccanismi di controllo (gate): uno decide quante informazioni nuove scrivere nello stato interno, uno decide quanto del passato dimenticare, e uno decide cosa comunicare all'uscita. Questa struttura permette alla LSTM di mantenere selettivamente le informazioni utili per molti passi senza problemi di stabilità durante il training.

In AG-PPO la LSTM è posizionata tra il feature extractor CNN e le teste di policy e value. L'agente può così ricordare nel corso dell'episodio quali posizioni della griglia ha già visitato e quali configurazioni di casse ha già raggiunto, informazioni che la griglia corrente da sola non contiene.

2.4 Large Language Model

2.4.1 Architettura e capacità

I Large Language Model (LLM) sono reti neurali con architettura Transformer [10] addestrate su enormi corpus testuali per predire il token successivo di una sequenza. Il Transformer elabora l'intera sequenza in ingresso in parallelo usando un meccanismo di self-attention: ogni token viene interpretato in relazione a tutti gli altri, pesando il contesto in modo appreso durante il training. I modelli decoder-only come GPT o Qwen3 generano testo un token alla volta, con la sequenza già prodotta che funge da contesto per il token successivo.

Dal training su testi diversissimi emergono capacità che non erano state supervisionate in modo esplicito: ragionamento logico, pianificazione simbolica e comprensione di

contesti strutturati come le regole di un gioco. Modelli open-source come la famiglia Qwen3 di Alibaba possono girare localmente su hardware consumer tramite Ollama, senza dipendenza da servizi cloud.

2.4.2 LLM come agenti decisionali

In questo progetto si usa `qwen3:1.4b-q4_K_M`: 14 miliardi di parametri quantizzati a 4 bit, circa 9 GB in memoria. Come agente decisionale, l’LLM riceve ad ogni step un prompt che descrive le regole di Sokoban, riporta la griglia in formato ASCII con una legenda dei simboli, elenca le posizioni correnti di giocatore, casse e target, e chiede una delle quattro direzioni di movimento. La risposta viene parsata cercando la prima parola valida tra *su*, *giu*, *sinistra*, *destra*.

Il flag `think=False` disabilita la catena di ragionamento interno di qwen3, riducendo la latenza da circa 2 secondi a circa 0.2 secondi per chiamata. Su migliaia di chiamate durante il training questo costo si accumula, ed è il motivo per cui l’LLM viene interrogato solo in contesti specifici e non a ogni step.

2.5 Curriculum Learning

Il Curriculum Learning [11] è una tecnica di training in cui i task vengono presentati all’agente in ordine crescente di difficoltà, invece di esporlo da subito alla distribuzione finale. L’analogia con l’apprendimento umano è intuitiva: nessuno impara a giocare a scacchi partendo dalle partite tra grandi maestri.

Per il Reinforcement Learning con reward sparsa il vantaggio è concreto. Con una sola cassa su griglia piccola, un agente che esplora a caso trova soluzioni abbastanza spesso da ricevere un segnale di training utile. Con quattro casse su griglia 10×10 , la probabilità di riuscire per caso è vicina a zero e il training non parte. Il curriculum costruisce la competenza gradualmente: l’agente impara a spostare una cassa, poi due, poi tre, e arriva alla variante a quattro casse con già una policy di base.

In questo progetto sono presenti due curriculum distinti. Quello del sistema 10×10 ha sei fasi (C0–C5): le prime quattro usano livelli generati proceduralmente con numero di casse crescente (da 1 a 4); le ultime due usano livelli del dataset

Boxoban con 4 casse. Quello del sistema 7×7 ha tre fasi (C0–C2) con livelli generati proceduralmente e al massimo 3 casse, senza dataset esterno. In entrambi i casi un meccanismo adattivo ripete la fase corrente se il solve rate non supera la soglia minima prevista, fino a un massimo di tre tentativi per fase.

2.6 Learning from Demonstrations

Il Learning from Demonstrations (LfD) affianca il training RL con traiettorie fornite da un agente esterno, spesso più esperto. Le dimostrazioni pre-riempiono il replay buffer con esempi di comportamento utile: nelle prime iterazioni il DQN impara da queste transizioni invece di partire dall'esplorazione casuale, che nelle fasi iniziali raramente produce episodi di successo.

In AG-LLM-GUIDE il dimostratore è l'LLM: per ogni fase del curriculum gioca 30 episodi autonomi, registrando ogni transizione (stato, azione, reward, stato successivo, fine episodio) e salvandola su disco. Il DQN viene poi addestrato con il replay buffer pre-riempito con queste dimostrazioni. Man mano che il training avanza, l'esperienza raccolta dall'agente stesso si mescola alle dimostrazioni LLM nel buffer. L'approccio si ispira a DQfD [12], che ha mostrato come dimostrazioni di qualità sufficiente accelerino la convergenza del DQN in ambienti con reward sparsa.

3.1 RL applicato ai videogiochi

Nel 2015 Mnih et al. [6] mostrarono che una rete neurale può imparare a giocare a 49 titoli Atari usando solo i pixel come input, senza alcuna conoscenza specifica del gioco. Prima di DQN nessun algoritmo ci era riuscito in modo credibile su task visivi ad alta dimensione: ogni gioco richiedeva rappresentazioni e regole costruite a mano. Quella dimostrazione cambiò il modo in cui il campo guardava all'RL applicato a problemi percettivi complessi.

Per la pianificazione a lungo termine, AlphaGo [13] mostrò che combinare RL con Monte Carlo Tree Search basta a battere i campioni mondiali di Go, un gioco dove la ricerca classica era praticamente cieca per l'ampiezza dell'albero. AlphaZero [14] andò oltre: stesso algoritmo, nessun dato di apertura umano, risultati analoghi su Go, scacchi e shogi imparando esclusivamente per self-play.

Una rete neurale che approssima funzioni di valore o policy, guidata da un algoritmo di ricerca o aggiornamento: è l'impostazione comune a entrambi i sistemi. Sokoban non ammette né la ricerca sull'albero di gioco né una reward densa, quindi i due lavori sono utili come termini di paragone ma non trasferibili a questo progetto.

3.2 Sokoban come benchmark per il RL

In Atari si perde una vita e si ricomincia. In Sokoban una mossa sbagliata può rendere un livello irrisolvibile per sempre, e spesso non si vede subito: il livello sembra ancora fattibile, ma la configurazione è già bloccata in un dead-end che nessuna sequenza di mosse potrà sciogliere. Trovare episodi di successo è difficile; capire dove si è andati storto lo è ancora di più.

Il lavoro di riferimento sulla variante a 4 casse è Guez et al. [3], che insieme al dataset Boxoban pubblica risultati sistematici su algoritmi model-free. L'architettura IMPALA+DRC (Deep Repeated ConvLSTM), addestrata per 2 miliardi di step su hardware distribuito, raggiunge circa il 99% di solve rate sull'insieme unfiltered. Questo progetto usa una singola GPU con circa 9 milioni di step per AG-PPO e 6 milioni per AG-DQN: un budget inferiore di circa due ordini di grandezza. Il confronto va tenuto in mente leggendo i risultati del Capitolo 7.

L'articolo documenta anche perché Sokoban sia più difficile di Atari per i metodi model-free: reward sparsa, orizzonte di pianificazione lungo, struttura del problema che non permette di recuperare dagli errori locali.

3.3 Curriculum Learning nei task combinatori

Bengio et al. [11] formalizzarono l'idea: esporre l'agente prima ai task semplici porta a convergenza più rapida e a soluzioni migliori rispetto a partire dall'inizio con la distribuzione completa. Per problemi con reward sparsa il guadagno è immediato. Con una sola cassa su griglia 7×7 l'esplorazione casuale produce abbastanza successi da generare un segnale di addestramento utile. Con quattro casse su griglia 10×10 la probabilità di riuscirci per caso è vicina a zero e il training non parte.

Il curriculum adattivo è stato applicato a task robotici come manipolazione di oggetti e locomozione bipede, e a giochi combinatori come Rubik's Cube. Il meccanismo comune è il criterio di avanzamento: l'agente passa alla fase successiva solo dopo aver superato una soglia di performance, non dopo un numero fisso di step. Trasferire prematuramente una policy ancora debole al task successivo non aiuta; spesso peggiora le cose rispetto ad aspettare.

3.4 LLM come agenti di pianificazione

Wei et al. [15] mostrarono con il chain-of-thought prompting che chiedere a un LLM di produrre passi intermedi di ragionamento prima della risposta finale migliora le prestazioni su task matematici e logici. ReAct [16] estese l'idea ai task interattivi: il modello alterna ragionamento e azioni nell'ambiente, correggendo la direzione in base al feedback ricevuto. Su HotpotQA e ALFWorld i risultati superano sia gli approcci puramente ragionativi che quelli puramente reattivi.

Su Sokoban il limite è strutturale. Il gioco richiede lookahead: bisogna prevedere dove finirà una cassa dopo più spinte consecutive e tenere traccia di configurazioni già irrisolvibili. Gli LLM lavorano su descrizioni testuali dello stato corrente, non su alberi di ricerca espliciti. Nella pratica producono mosse localmente sensate ma globalmente controproducenti, senza accorgersi di stare costruendo un dead-end. Il flag `think=False` usato in questo progetto riduce la latenza da circa 2 secondi a circa 0.2 secondi per chiamata, rinunciando alla catena di pensiero del modello: un compromesso necessario per rendere fattibile il training, ma che non migliora la qualità delle decisioni.

3.5 Integrazione LLM e Reinforcement Learning

InstructGPT [17] ha reso sistematica l'idea di usare un reward model addestrato su preferenze umane per guidare un LLM tramite RL. Il reward model valuta coppie di risposte e l'agente viene aggiornato per massimizzare quel segnale. La struttura si può invertire: l'LLM valuta le azioni di un agente RL e fornisce un segnale di reward aggiuntivo a quello dell'ambiente.

Questo progetto usa questa idea in AG-LLM-REW, dove l'LLM assegna una reward pesata ($\lambda = 0.3$) a ogni push di cassa. AG-LLM-GUIDE adotta un approccio diverso: l'LLM interviene solo durante la raccolta delle dimostrazioni, ispirandosi a DQfD [12]. A inference time l'LLM non serve più, e la performance finale dipende dalla qualità media delle dimostrazioni raccolte offline, non dalla qualità di ogni singola chiamata durante la valutazione.

Il gioco Sokoban: ambiente e dataset

4.1 Regole e meccaniche di gioco

Sokoban si svolge su una griglia bidimensionale. Ogni cella può contenere un muro, un pavimento vuoto, un bersaglio, una cassa, una cassa già posizionata su bersaglio, il giocatore o il giocatore sopra un bersaglio. In ogni turno il giocatore si sposta in una delle quattro direzioni ortogonali. Se la destinazione è libera, ci va. Se contiene una cassa, la spinge nella stessa direzione, ma solo se la cella immediatamente oltre è libera (pavimento o bersaglio vuoto). Non si può tirare una cassa, spingere due casse consecutive o attraversare un muro.

La vittoria si raggiunge quando tutte le casse si trovano sui bersagli. Non c'è corrispondenza obbligatoria tra cassa e bersaglio specifico: qualsiasi combinazione conta. La condizione viene verificata semplicemente controllando che non esistano casse libere nella griglia.

Ciò che rende il problema difficile è l'irreversibilità delle spinte. Una cassa finita in un angolo tra due muri non si recupera più: è lì per sempre. Questi dead-end non sono sempre visibili osservando lo stato corrente; molti si formano solo dopo una sequenza di mosse apparentemente corrette, senza che nello stato attuale ci sia alcun segnale che qualcosa sia andato storto.

4.2 Complessità computazionale

Culberson dimostrò nel 1997 che Sokoban è PSPACE-complete [1]. La classe PSPACE contiene NP, il che significa che non solo non si conosce alcun algoritmo efficiente per trovare soluzioni, ma verificare se una configurazione è irrisolvibile è altrettanto difficile.

Le implicazioni pratiche per il training con RL sono concrete. L'esplorazione casuale non trova soluzioni: con più di una o due casse la probabilità di successo casuale è vicina a zero. La ricerca sistematica nello spazio degli stati non è percorribile: con 4 casse su griglia 10×10 lo spazio supera i 10^8 stati. Il segnale di reward è sparso per costruzione, perché l'agente può eseguire decine di mosse corrette senza mai ricevere feedback positivo. Il curriculum learning adottato in questo progetto nasce proprio da questi tre vincoli.

4.3 Dataset DeepMind Boxoban

DeepMind ha reso pubblico nel 2019, insieme al lavoro di Guez et al. [3], un dataset di livelli Sokoban chiamato Boxoban, rilasciato su GitHub con licenza Apache 2.0. Il dataset ha tre sottoinsiemi: *unfiltered* (circa 900.000 livelli di training e circa 1.000 di test), *medium* (circa 500.000 livelli filtrati) e *hard* (circa 3.332 livelli). Tutti hanno griglia 10×10 e 4 casse.

Il formato è testuale ASCII, con un carattere per cella: # per muro, spazio per pavimento, . per bersaglio, \$ per cassa, * per cassa su bersaglio, @ per giocatore, + per giocatore su bersaglio. Il caricatore (`CaricatoreLivelli`) legge i file in modo lazy e converte ogni riga testuale in una matrice NumPy int8, scartando i livelli che non superano la validazione strutturale.

4.4 Ambiente custom con Gymnasium (sistema 10×10)

L'ambiente estende `gymnasium.Env`, quindi è compatibile con Stable Baselines 3 e con il wrapper `SubprocVecEnv` per la parallelizzazione su più processi distinti.

4.4.1 Rappresentazione dello stato

Lo stato della griglia usa 8 valori interi distinti:

Valore	Elemento	Simbolo Boxoban
0	MURO	#
1	PAVIMENTO	(spazio)
2	TARGET	.
3	CASSA	\$
4	CASSA_SU_TARGET	*
5	GIOCATORE	@
6	GIOCATORE_SU_TARGET	+
7	PADDING	—

Nel sistema 10×10 , l'observation space è `Box(0.0, 7.0, shape=(10, 10), dtype=float32)`. Le griglie di gioco più piccole di 10×10 vengono centrate in questo frame con valore di padding 7, distinto da MURO (0) e da tutte le celle di gioco (1–6): la CNN può così imparare feature che ignorano il bordo artificiale. Il sistema 7×7 adotta invece un observation space nativo, descritto nella Sezione 4.5. Internamente la griglia è mantenuta come array NumPy int8 per efficienza; la conversione in float32 avviene solo alla restituzione dell'osservazione.

4.4.2 Spazio delle azioni

Lo spazio delle azioni è `Discrete(4)`: 0 = su, 1 = giù, 2 = sinistra, 3 = destra. Le mosse non valide (verso un muro, o con cassa non spingibile) non modificano la griglia: `applica_mossa` restituisce una copia invariata con `mossa_eseguita=False`. Non viene applicata nessuna penalità aggiuntiva per i tentativi non validi.

4.4.3 Sistema di ricompense

La reward a ogni step è la somma di cinque componenti:

- -0.005 per ogni azione, valida o meno (penalità di step);

- +1.0 per ogni cassa appena posizionata su target, -1.0 per ogni cassa appena rimossa;
- +10.0 al completamento del livello (`terminated=True`);
- reward shaping basato sulla distanza Manhattan ottimale tra casse e target (algoritmo Ungherese), scalato per $\lambda_M = 0.3$: la differenza di distanza tra il passo precedente e il passo corrente, positiva se le casse si avvicinano ai target;
- reward shaping basato sulla distanza del giocatore dalla cassa più vicina, scalato per $\lambda_P = 0.1$: incentiva la navigazione verso le casse prima di spingerle.

Le ultime due componenti sono delta-based (differenza tra passi consecutivi): non dipendono dalla posizione assoluta, solo dalla variazione a ogni mossa. Sono abilitate in entrambi i sistemi con i parametri `scala_manhattan=0.3` e `scala_player_box=0.1`: nel sistema 10×10 tramite `src/sistema_10x10/config.py`, nel sistema 7×7 tramite i valori hardcoded in `sokoban_gym_7x7.py`.

Il limite massimo è 120 step per episodio di default, variabile per fase di curriculum. Superato il limite l'episodio è troncato (`truncated=True`, `terminated=False`): solo gli episodi con `terminated=True` contano come risolti nella valutazione finale.

La penalità di step è arrivata a -0.005 gradualmente: nelle prime versioni era -0.1 , troppo severa per soluzioni lunghe; ridotta a -0.01 in una revisione intermedia, poi portata al valore finale di -0.005 per permettere all'agente di esplorare sequenze più lunghe senza che la penalità cumulativa rendesse inutile qualsiasi risoluzione.

4.5 Sistema semplificato 7×7

4.5.1 Motivazioni

Su singola GPU i livelli Boxoban a 4 casse su griglia 10×10 non erano accessibili con il budget di training disponibile. Per avere un secondo sistema in cui almeno alcune fasi del curriculum producessero risoluzioni concrete, è stato implementato un ambiente con griglia nativa 7×7 e al massimo 3 casse, con livelli generati proceduralmente invece che caricati da dataset.

L'observation space è nativo 7×7 : `Box(0.0, 6.0, shape=(7,7), dtype=float32)`, senza padding. Il valore massimo è 6.0 perché il valore di padding (7) non esiste in questo sistema: la griglia è già della dimensione corretta. La compatibilità con la stessa `SokobanCNN` è garantita dal fatto che la dimensione del flatten layer viene calcolata dinamicamente con una forward pass su tensore dummy alla costruzione del modello, per cui la stessa architettura funziona sia su input $(1, 10, 10)$ che su $(1, 7, 7)$. Il wrapper `AggiuntaCanale` aggiunge la dimensione di canale anche per il sistema 7×7 : $(7, 7) \rightarrow (1, 7, 7)$.

4.5.2 Generatore di livelli e verifica di risolubilità

`GeneratoreLivelli` costruisce livelli procedurali in quattro passi: genera una griglia con muri perimetrali e interno a pavimento, aggiunge muri interni casuali fino al 20% delle celle interne, posiziona casualmente giocatore, casse e target tra le celle libere, verifica la risolubilità. Se il livello non supera la verifica, riprova fino a 500 volte prima di sollevare un'eccezione. La strategia di verifica dipende dalla dimensione stimata dello spazio degli stati, calcolata come `celle_interne(n_casse+1)`:

- **BFS esatto** (stima ≤ 50.000 stati): visita l'intero spazio degli stati; se esiste una soluzione la trova, altrimenti il livello viene scartato. Risolubilità garantita.
- **Euristica dead-corner** (stima > 50.000 stati): controlla che nessuna cassa si trovi in un angolo formato da due muri perpendicolari senza un target in quell'angolo, e che il giocatore possa raggiungere almeno una cassa. Condizione necessaria, non sufficiente: possono esistere livelli accettati che non sono risolubili.

Per la griglia 10×10 (celle interne $8 \times 8 = 64$):

- C0 (1 cassa): $64^2 = 4.096$ stati $\rightarrow$ BFS esatto, risolubilità garantita;
- C1 (2 casse): $64^3 = 262.144$ stati $\rightarrow$ euristica, possibili falsi positivi;
- C2 (3 casse): $64^4 = 16.777.216$ stati $\rightarrow$ euristica;
- C3 (4 casse): $64^5 = 1.073.741.824$ stati $\rightarrow$ euristica.

Solo C0 ha risolubilità garantita. Per C1–C3 l'euristica dead-corner è condizione necessaria ma non sufficiente: alcuni livelli accettati potrebbero non avere soluzione.

Per la griglia 7×7 (celle interne $5 \times 5 = 25$):

- C0 (1 cassa): $25^2 = 625$ stati $\rightarrow$ BFS esatto, risolubilità garantita;
- C1 (2 casse): $25^3 = 15.625$ stati $\rightarrow$ BFS esatto, risolubilità garantita;
- C2 (3 casse): $25^4 = 390.625$ stati $\rightarrow$ euristica, possibili falsi positivi.

Nel sistema 7×7 anche C1 rientra nel BFS esatto: con 25 celle interne la stima resta ampiamente sotto la soglia di 50.000.

5.1 Approccio generale al problema

Il lavoro ha seguito quattro fasi in sequenza: costruzione e test dell'ambiente, addestramento degli agenti RL puri con curriculum learning, integrazione dell'LLM nelle tre modalità richieste, valutazione comparativa finale.

Il sistema è stato duplicato e prevede:

- ambiente 10×10 principale con livelli dal dataset DeepMind Boxoban come benchmark finale
- e un ambiente 7×7 secondario con livelli generati proceduralmente per avere un secondo piano di osservazione dove il curriculum produce risoluzioni concrete.

I due sistemi usano la stessa CNN e la stessa struttura di addestramento.

5.2 Curriculum learning: struttura e fasi

5.2.1 Sistema 10×10 : curriculum adattivo a sei fasi

Il curriculum di addestramento si articola in 6 fasi su una griglia di dimensioni fisse pari a 10×10 . Questa scelta costruttiva si è resa necessaria poiché, nelle versioni precedenti (v1–v7), l’uso di griglie di dimensioni crescenti con *padding* centrato induceva la CNN ad apprendere l’offset del *padding* anziché la reale struttura del puzzle. Mantenere la griglia costante per tutto il *training* ha permesso di risolvere l’anomalia.

Fase	Dataset	Max step	Soglia	Budget PPO / DQN
C0	1 cassa generata	120	15%	600K / 400K
C1	2 casse generate	150	10%	1M / 700K
C2	3 casse generate	200	5%	1.5M / 1M
C3	4 casse generate	250	3%	2M / 1.4M
C4	Boxoban medium	300	2%	2M / 1.2M
C5	Boxoban unfiltered	300	—	2M / 1.3M

Il budget totale nominale è 9,1M step per AG-PPO e 6M per AG-DQN. I valori sono scelte progettuali calibrate fase per fase: PPO usa 8 ambienti paralleli via `SubprocVecEnv` e DQN un ambiente singolo, ma i budget contano le interazioni ambientali totali allo stesso modo per entrambi.

5.2.2 Sistema 7×7 : curriculum semplificato

Il sistema 7×7 usa tre fasi (C0: 1 cassa, C1: 2 casse, C2: 3 casse) con gli stessi meccanismi adattivi del sistema 10×10 . Tutti i livelli sono generati proceduralmente: BFS esatto per C0 e C1, euristica dead-corner per C2 (vedi Sezione 4.5.2).

Fase	N. casse	Max step	Soglia	Budget PPO / DQN
C0	1 generata	80	50%	400K / 280K
C1	2 generate	120	15%	700K / 500K
C2	3 generate	180	—	1M / 700K

Il budget totale nominale è 2,1 M step per AG-PPO e 1,48 M per AG-DQN. Le soglie del sistema 7×7 sono più alte di quelle del sistema 10×10 per le fasi corrispondenti (50% vs 15% su C0) perché il problema è più semplice: la griglia più piccola riduce lo spazio degli stati e rende l'esplorazione casuale più produttiva, quindi si può esigere una performance più alta prima di avanzare alla fase successiva.

5.2.3 Curriculum adattivo e gestione del budget

L'avanzamento tra fasi dipende dal solve rate misurato, non dal numero di step completati. Ogni fase valuta la policy su 100 episodi ogni 20.000 interazioni ambientali circa. Se la soglia non è raggiunta a fine budget, la fase si ripete con lo stesso budget. Il numero massimo di ripetizioni aggiuntive è 2, quindi ogni fase può consumare al massimo $3 \times$ il budget nominale. Se la soglia non viene raggiunta neanche alla terza esecuzione, il training avanza comunque.

Il modello migliore per ciascuna fase viene salvato tramite `EvalCallback`. Al cambio di fase si carica questo checkpoint invece del modello all'ultimo step, per contenere la perdita di performance al cambio di distribuzione tra fasi.

5.3 Architettura della rete neurale convoluzionale

Tutti gli agenti RL (AG-PPO, AG-DQN, AG-LLM-GUIDE, AG-LLM-REW) usano la stessa rete convoluzionale, chiamata `SokobanCNN`, per trasformare la griglia di gioco in una rappresentazione numerica su cui l'algoritmo RL può ragionare.

La griglia arriva come matrice di valori interi (da 0 a 7), che vengono prima normalizzati nell'intervallo $[0, 1]$. Tre strati convoluzionali analizzano progressivamente la struttura spaziale: il primo rileva pattern locali su ogni cella e i suoi vicini immediati (angoli di muri, posizioni di casse); il secondo aumenta l'astrazione riducendo la risoluzione della mappa a metà; il terzo raffina le feature estratte. Il risultato viene poi appiattito in un singolo vettore di numeri, 1600 valori per il sistema 10×10 e 1024 per quello 7×7 , e compresso da uno strato lineare finale a 256 valori. Questi 256 numeri sono quello che l'agente RL riceve come descrizione della situazione di gioco ad ogni step.

La stessa architettura funziona su entrambe le griglie senza modifiche: la dimensione del vettore intermedio viene calcolata automaticamente alla costruzione del modello, adattandosi alla dimensione dell'input.

5.4 Design dei cinque agenti

5.4.1 AG-PPO: baseline con memoria ricorrente

AG-PPO usa `RecurrentPPO` da `sb3-contrib` con policy `CnnLstmPolicy`. La scelta di `RecurrentPPO` rispetto a PPO standard nasce da una ragione pratica: Sokoban richiede pianificazione su più mosse, e la LSTM integrata nella policy può ricordare la storia degli step dell'episodio corrente, cosa che un MLP non può fare.

Iperparametri:

- `learning_rate`: 3×10^{-4}
- `n_steps`: 2048
- `batch_size`: 64
- `n_epochs`: 10
- γ : 0.99
- `gae_lambda`: 0.95
- `clip_range`: 0.2
- Ambienti paralleli: 8
- Il coefficiente entropico (`ent_coef`) sale da 0.01 nelle fasi iniziali a 0.03 nelle ultime due per mantenere l'esplorazione man mano che la distribuzione dei livelli diventa più difficile (nel sistema 7×7 , con sole tre fasi, il massimo raggiunto è 0.02).

5.4.2 AG-DQN: apprendimento off-policy

AG-DQN usa DQN di SB3 con CnnPolicy e la stessa SokobanCNN.

Iperparametri:

- `learning_rate`: 10^{-4}
- `buffer_size`: 100.000
- `learning_starts`: 1.000
- `batch_size`: 32
- γ : 0.99
- `exploration_fraction`: 0.15
- `exploration_final_eps`: 0.05
- `target_update_interval`: 1.000
- Ambiente: singolo ($N_{ENVs} = 1$)

Il replay buffer conserva le transizioni raccolte nelle fasi precedenti, che continuano a partecipare agli aggiornamenti anche nelle fasi successive. Su curriculum a distribuzione variabile questo è un vantaggio strutturale rispetto agli algoritmi on-policy.

5.4.3 AG-LLM-ACT: LLM come controllore diretto

AG-LLM-ACT non ha training RL. Ad ogni step la griglia viene convertita in una rappresentazione ASCII e inviata all'LLM come prompt. Il modello risponde con una parola tra {su, giu, sinistra, destra}, mappata direttamente all'azione. Il flag `think=False` disabilita il chain-of-thought interno di qwen3, riducendo la latenza da circa 2 secondi a circa 0.2 secondi per chiamata. La valutazione copre 100 episodi per fase, comparabile con gli agenti RL.

5.4.4 AG-LLM-GUIDE: learning from demonstrations

AG-LLM-GUIDE combina due fasi per ogni livello del curriculum. Prima il LLM gioca 30 episodi autonomamente, registrando tutte le transizioni (*obs*, *action*, *reward*, *next_obs*, *done*). Poi il DQN viene addestrato con il replay buffer pre-riempito con queste dimostrazioni, in modo che le prime iterazioni partano da transizioni LLM invece che da esplorazione casuale.

La struttura DQN è identica ad AG-DQN: stessi iperparametri, stesso curriculum adattivo, stesse soglie. A inference time l'LLM non è necessario: agisce solo il DQN.

5.4.5 AG-LLM-REW: LLM come reward model

AG-LLM-REW usa `RecurrentPPO` con la stessa architettura di AG-PPO (`CnnLstmPolicy`, stessi iperparametri), con due differenze operative:

- Il numero di ambienti paralleli è 1 invece di 8, ciò è dovuto a Ollama che non supporta chiamate concorrenti dallo stesso processo e di conseguenza il training è sequenziale;
- Il budget di step per fase resta identico ad AG-PPO (stesso `FASI_CURRICULUM_V9`, 9,1 M totali), ma viene percorso in una sola passata per fase, senza ripetizioni adattive: così il confronto con AG-PPO è equo a parità di interazioni ambientali e l'unica vera differenza resta il singolo ambiente. Il costo aggiuntivo si paga in tempo di calcolo, per la latenza di Ollama.

Il wrapper `Gymnasium (RicompensaLLM)` intercetta ogni `step()` e può aggiungere un contributo di reward dall'LLM. L'LLM viene chiamato solo quando `info['cassa_spostata']` è True, cioè quando una cassa è stata effettivamente spinta: circa il 5% degli step totali.

L'LLM riceve la griglia prima e dopo la spinta e risponde con un giudizio da 0 a 3: 0 significa mossa sbagliata o bloccante, 1 neutro, 2 buona, 3 ottima. Questo giudizio viene convertito in un piccolo contributo di reward, da -0.15 per il punteggio peggiore a $+0.15$ per il migliore, e sommato alla reward standard dell'ambiente. Se la stessa configurazione e la stessa azione si ripresentano, il giudizio già calcolato viene riutilizzato senza richiamare l'LLM. A inference time l'LLM non serve.

5.5 Integrazione del Large Language Model

5.5.1 Scelta del modello: da llama3.1 a qwen3:14b

Il primo modello testato è stato `llama3.1:8b` via Ollama. Solve rate 0% su tutti i livelli, compreso C0 a una sola cassa: il modello produceva quasi esclusivamente wall-collision e l'output JSON era spesso malformato, richiedendo un fallback sull'azione casuale. Di fatto, il modello non era utilizzabile.

Il cambio a `qwen3:14b-q4_K_M` ha risolto entrambi i problemi: 9% di solve rate su C0 del sistema 7×7 , zero fallback sull'output JSON, nessuna wall-collision sistematica. La quantizzazione `q4_K_M` (4 bit per peso) riduce l'occupazione in VRAM: il modello gira interamente su GPU senza split CPU/GPU, nel sistema specifico è stata utilizzata una GeForce MSI RTX 4070 Gaming X Trio con 12GB di VRAM dedicata.

5.5.2 Progettazione del prompt

I prompt sono stati progettati per essere il più brevi possibile, perché ogni token in input aumenta la latenza di tokenizzazione e prefill. Il prompt per AG-LLM-ACT si attesta intorno a 150 token; quello per AG-LLM-REW, che deve mostrare due griglie, arriva a circa 260 token.

Prompt AG-LLM-ACT. Ad ogni step l'LLM riceve la griglia corrente in ASCII, la legenda dei simboli, il contatore casse/bersagli, gli step rimasti, e la richiesta esplicita di rispondere con una sola parola tra le quattro azioni. Un esempio reale con una cassa (C0):

```
Sokoban: spingi le casse sui target.
# muro  @ tu  $ cassa  . target  * cassa su target

#####
#           #
#   .       #
#   $       #
```

```
#   @   #
#       #
#####
#####
#####
#####
```

Casse su target: 0/1 | Step rimasti: 118/120

Rispondi con UNA SOLA parola, senza spiegazioni: su giu sinistra destra
Mossa:

La risposta attesa è una parola sola (su). Le ultime tre righe di # sono il padding che riporta la griglia interna a 10×10 .

Prompt AG-LLM-REW. Quando una cassa viene spinta, l’LLM riceve le due griglie (prima e dopo la mossa), il nome dell’azione e il contatore casse, e deve rispondere con un solo numero intero. Un esempio con la cassa appena posizionata sul bersaglio:

Sokoban: valuta l'azione eseguita confrontando prima e dopo.
muro @ tu \$ cassa . target * cassa su target

Prima:

```
#####
#       #
#   .   #
#   $   #
#   @   #
#       #
#####
```

Dopo:

```
#####
#       #
#   *   #
#   @   #
```

```
#           #
#           #
#####
```

Azione eseguita: SU

Casse su target: 1/1

Rispondi con UN SOLO numero intero, nessuna spiegazione:

0=azione sbagliata/bloccante 1=neutro 2=buono 3=ottimo

Punteggio:

La risposta attesa è 3 (ottimo): la cassa è finita sul bersaglio. Il punteggio 3 produce un contributo di reward +0.15.

5.5.3 Gestione della latenza

Il client LLM (`ClienteLLM`) usa `http.client` con connessione HTTP persistente (keep-alive) verso il server Ollama in locale. Rispetto a `urllib.request` con nuova connessione TCP a ogni chiamata (circa 2.3 s), la connessione persistente porta la latenza a 0.14–0.21 s a regime.

Al momento della costruzione del client viene eseguito un warm-up: il modello viene forzato a caricarsi con `num_gpu=999` e `num_ctx=512`. Il contesto ridotto a 512 token (invece dei 4096 di default) riduce la KV cache da circa 4 GB a circa 0.1 GB, permettendo al modello di stare interamente in VRAM. Insieme al flag `think=False` che salta il chain-of-thought interno, queste ottimizzazioni riducono la latenza media di un fattore circa 10 rispetto alla configurazione predefinita.

Implementazione

6.1 Stack tecnologico

Il progetto è scritto interamente in Python 3.13. Le due librerie centrali sono Gymnasium 1.2.3, che definisce l'interfaccia standard degli ambienti con cui SB3 si aspetta di lavorare, e Stable Baselines 3 2.7.1 con sb3-contrib, che fornisce le implementazioni di PPO, RecurrentPPO e DQN già integrate con il sistema di callback, logging e salvataggio dei modelli. Il training gira su GPU tramite PyTorch 2.10.0 con supporto CUDA 12.6 e una GeForce RTX 4070 da 12 GB di VRAM.

Per l'LLM non si usa nessuna libreria aggiuntiva: il client è scritto direttamente sopra `http.client` della libreria standard Python, per avere il controllo totale sulla connessione persistente che riduce la latenza. L'unica dipendenza esterna non banale è `scipy.optimize.linear_sum_assignment`, usata per l'algoritmo Ungherese nel reward shaping Manhattan.

Il codice è organizzato in cartelle che separano nettamente i componenti:

- `src/core/ambiente/`: logica di gioco e ambiente Gymnasium principale;
- `src/core/agenti_llm/`: i tre agenti LLM (ACT, GUIDE, REW); PPO e DQN sono configurati e addestrati direttamente in `src/sistema_10x10/train_ppo.py`

`etrain_dqn.py`;

- `src/core/llm/`: client LLM e generazione dei prompt;
- `src/sistema_10x10/`: script di training, valutazione e configurazione centralizzata;
- `src/sistema_7x7/`: sistema 7×7 con ambiente e configurazione propri, che riutilizza `src/core/` (CNN, agenti LLM, client);
- `tests/`: suite di test unitari.

6.2 Ambiente di simulazione

6.2.1 Il sistema 10×10

L'ambiente principale è in `sokoban_gym.py`, ma la logica vera è distribuita su più file con responsabilità distinte. Questa separazione non è formale: serve a rendere possibile il testing.

`game_logic.py` contiene le funzioni che applicano le mosse, controllano la vittoria e contano le caselle posizionate. Sono funzioni pure che ricevono una griglia e restituiscono una griglia nuova, senza stato interno. Questo significa che si testano passando una matrice a mano, senza bisogno di avviare l'ambiente completo.

`level_loader.py` gestisce il caricamento dei livelli Boxoban. I file del dataset occupano diversi gigabyte e contengono centinaia di migliaia di livelli: caricarli tutti all'avvio sarebbe lento e inutile. Il caricatore li legge su richiesta, li converte in matrici NumPy int8 e li valida prima di metterli in uso.

`reward.py` calcola la reward ad ogni step combinando le cinque componenti descritte nel Capitolo 4. Il reward shaping Manhattan usa l'algoritmo Ungherese per trovare l'abbinamento ottimale tra caselle e bersagli, che non è necessariamente la coppia più vicina ma quella che minimizza la distanza totale.

`sokoban_gym.py` orchestra tutto questo nell'interfaccia Gymnasium, gestendo `reset`, `step`, `rendering` e le informazioni aggiuntive che gli agenti LLM richiedono. `renderer.py` serve solo per le demo interattive e non viene caricato durante il training.

L'ambiente è compatibile con `SubprocVecEnv` di SB3, che lancia otto copie dell'ambiente in processi separati per il training parallelo di AG-PPO. Ogni processo gestisce il proprio `SokobanEnv` indipendente; non c'è stato condiviso tra processi.

6.2.2 Il sistema 7×7

Il sistema 7×7 è implementato in `src/sistema_7x7/`, senza toccare nessun file di `src/core/ambiente/`. Ha la propria versione dell'ambiente (`sokoban_gym_7x7.py`), la propria configurazione, i propri script di training e cartelle separate per modelli (`artifacts/models/7x7/`) e log (`artifacts/logs/7x7/`).

La differenza tecnica principale riguarda l'observation space: nel sistema 10×10 la griglia viene sempre restituita come matrice 10×10, usando il valore 7 come padding per le griglie più piccole. Il sistema 7×7 usa invece una griglia nativa 7×7, quindi l'observation space è `Box(0.0, 6.0, shape=(7,7), dtype=float32)` e il valore di padding non esiste. Questa è la stessa correzione che ha risolto il fallimento del curriculum nelle versioni precedenti del sistema 10×10: la CNN non impara feature legate a un offset artificiale se non c'è nessun offset.

6.2.3 Test unitari

Il progetto include 64 test pytest, distribuiti in tre file. I 18 test di `test_game_logic.py` verificano la logica pura: che le mosse vengano applicate correttamente, che le situazioni di dead-end vengano riconosciute, che la vittoria venga rilevata al momento giusto. I 18 test di `test_env.py` verificano l'ambiente Gymnasium completo: il reset, i cicli step/reward, la compatibilità con SB3. I 28 test di `test_llm_integration.py` verificano il client LLM e il parsing delle risposte, usando per lo più risposte preregistrate invece di chiamare Ollama; il numero più alto rispetto agli altri due file è dovuto ai test parametrizzati su `parsifica_azione`, che coprono dieci varianti di formato della risposta LLM. Dei 64 test complessivi, 61 passano in qualsiasi ambiente, senza accesso al dataset Boxoban e senza un server Ollama attivo; i 3 test che istanziano davvero il `ClienteLLM` contro il server locale vengono saltati automaticamente quando Ollama non è in esecuzione, e passano quando il server è attivo.

6.3 CNN e adattamento all'interfaccia SB3

`SokobanCNN` estende `BaseFeaturesExtractor`, la classe base che SB3 usa per iniettare estrattori di feature personalizzati nelle policy. Basta passarla nei `policy_kwargs` alla creazione del modello e SB3 la usa automaticamente, senza modifiche al codice di training.

C'è però un problema di formato: l'ambiente restituisce osservazioni di forma (H, W) , cioè una matrice bidimensionale. Le policy CNN di SB3 si aspettano tensori *channels-first*, cioè (C, H, W) , dove C è il numero di canali. Siccome la griglia ha un solo canale (non è un'immagine RGB), serve aggiungere una dimensione in testa. Il wrapper `AggiuntaCanale` fa esattamente questo: trasforma ogni osservazione da (H, W) a $(1, H, W)$. Nella catena di elaborazione, ogni episodio passa prima attraverso `SokobanEnv`, poi `AggiuntaCanale`, poi `Monitor` (che raccoglie le statistiche per `TensorBoard`), poi `VecEnv` per la parallelizzazione.

6.4 Training di AG-PPO

Lo script `src/sistema_10x10/train_ppo.py` implementa il curriculum adattivo come un ciclo Python che gira sulle sei fasi in sequenza. Per ogni fase, crea o carica il modello `RecurrentPPO` con la `SokobanCNN` come estrattore di feature, allena per il budget nominale di step, poi decide se passare alla fase successiva o ripetere.

La decisione si basa sul solve rate massimo raggiunto durante la fase appena completata. Qui c'è un dettaglio non ovvio: con il reward shaping attivo, un episodio non risolto può accumulare una reward cumulativa positiva se le casse si avvicinano abbastanza ai bersagli. Il flag `terminated`, che distinguerebbe un vero completamento da un episodio troncato, non è direttamente accessibile dai log di `EvalCallback`. Il curriculum usa quindi una soglia sulla reward episodica totale: 9.0. Un episodio effettivamente risolto ha sempre $\text{reward} \geq 9.5$ grazie al bonus di completamento +10, mentre uno non risolto non arriva a 9.0 con i parametri usati, anche nel caso migliore di reward shaping.

`EvalCallback` valuta la policy su 100 episodi ogni 20.000 interazioni ambientali circa e salva il modello migliore nella sottocartella `best/`. Al cambio di fase lo script

carica questo checkpoint, non il modello all'ultimo step: in molte fasi il modello ha performance migliori a metà training che alla fine, perché la distribuzione dei livelli non è stazionaria all'interno di una stessa fase.

6.5 Training di AG-DQN

Il curriculum adattivo di `src/sistema_10x10/train_dqn.py` è strutturato come quello di AG-PPO: stesse soglie, stessa logica di avanzamento, stesso salvataggio dei checkpoint. Le differenze sono nella natura dell'algoritmo.

DQN non supporta ambienti paralleli, quindi il training avviene su un singolo ambiente. Il replay buffer da 100.000 transizioni non viene svuotato tra una fase e l'altra: quando la distribuzione dei livelli cambia passando da C0 a C1, le esperienze raccolte su C0 restano nel buffer e continuano a partecipare agli aggiornamenti. Questo può sembrare un rischio, ma in realtà risulta essere un vantaggio: le competenze di base apprese nelle fasi precedenti rimangono disponibili come punto di partenza per le fasi successive.

6.6 Valutazione di AG-LLM-ACT

AG-LLM-ACT non ha training. Lo script `src/sistema_10x10/train_llm_act.py` esegue 100 episodi per fase, chiama l'LLM ad ogni step e salva tutte le metriche in formato JSON. Per il sistema 7×7 esiste una variante identica in `src/sistema_7x7/train_llm_act_7x7.py`.

Ad ogni step la griglia viene convertita in ASCII e inviata come prompt. La risposta dell'LLM viene normalizzata in minuscolo, liberata dalla punteggiatura e cercata la prima parola valida tra `su`, `giu`, `sinistra`, `destra`. Se non si trova nessuna corrispondenza, si sceglie un'azione casuale. Con qwen3:14b questo fallback non si è mai verificato nei 100 episodi valutati.

6.7 Training di AG-LLM-GUIDE

`AgenteAgLLMGuide` (in `src/core/agenti_llm/llm_guide_agent.py`) gestisce le due fasi del processo per ogni livello del curriculum.

Nella prima fase, l'LLM gioca 30 episodi autonomamente. Il prompt include, oltre alla griglia ASCII, le coordinate esplicite di giocatore, casse e bersagli in formato riga/colonna, e le ultime cinque mosse eseguite. Questo storico riduce i loop in cui l'LLM ripete la stessa mossa non valida: se vede che ha fatto su cinque volte di fila senza risultato, tende a provare altro. Le transizioni vengono salvate con osservazioni già in formato `(1, 10, 10)`, pronte per il replay buffer.

Nella seconda fase, le transizioni vengono caricate nel replay buffer del DQN tramite `buf.add()`. C'è un dettaglio di formato: SB3 con `N_ENVS=1` si aspetta osservazioni di shape `(1, 1, 10, 10)`, con una dimensione batch aggiuntiva in testa. Le osservazioni salvate nella prima fase hanno shape `(1, 10, 10)`, quindi si aggiunge la dimensione mancante con `np.expand_dims` prima di inserirle. Dopo il pre-fill il training DQN procede normalmente: man mano che l'agente raccoglie esperienza propria, questa si mescola alle dimostrazioni nel buffer.

6.8 Training di AG-LLM-REW

La struttura di `src/sistema_10x10/train_ppo_llm_rew.py` è la stessa di AG-PPO, con due differenze operative che derivano entrambe dall'uso di Ollama durante il training.

Ollama non gestisce chiamate concorrenti dallo stesso processo: se si usassero otto ambienti paralleli, le chiamate all'LLM si bloccherebbero in coda. Per questo AG-LLM-REW usa un solo ambiente. Il budget di step per fase resta però identico a quello di AG-PPO: lo stesso `FASI_CURRICULUM_V9`, da 600K step in C0 fino a 2M nelle fasi Boxoban, per un totale di circa 9,1M step. Mantenere lo stesso numero di interazioni ambientali rende il confronto con AG-PPO equo, a parità di esperienza raccolta; il costo si paga in tempo di calcolo, perché con un solo ambiente e le chiamate all'LLM il training è circa undici volte più lento e va eseguito in una sessione dedicata.

`RicompensaLLM` (in `src/core/agenti_llm/llm_reward_agent.py`) è un wrapper `Gymnasium` che intercetta ogni chiamata a `step()` e, se una cassa è stata spinta, chiede all'LLM di valutare la mossa. Per poter confrontare la griglia prima e dopo la spinta, il wrapper salva l'osservazione corrente ad ogni `reset()` e `step()`: quando arriva la richiesta di valutazione, la griglia pre-mossa è già disponibile senza chiamate aggiuntive all'ambiente.

Le valutazioni vengono messe in cache usando come chiave la coppia (griglia pre-mossa, azione eseguita). Se la stessa configurazione si ripresenta con la stessa azione, il giudizio già calcolato viene riutilizzato senza richiamare l'LLM. Questo succede più spesso di quanto si pensi: un agente nelle fasi iniziali del training tende a tornare spesso nelle stesse configurazioni.

6.9 Client LLM

`ClienteLLM` in `src/core/llm/llm_client.py` gestisce la comunicazione con Ollama: usa `http.client.HTTPConnection` con connessione persistente. Aprire una nuova connessione TCP a ogni chiamata ha un costo fisso di circa 2.3 secondi; tenerne aperta una sola e riusarla abbassa la latenza a 0.14–0.21 secondi a regime. La connessione viene aperta una volta sola al costruttore e mantenuta per tutta la durata del training.

Al momento della prima connessione il client esegue un warm-up: manda una richiesta con `num_gpu=999` per forzare il caricamento di tutti i layer del modello sulla GPU, e con `num_ctx=512` per limitare la KV cache. Il contesto predefinito di Ollama è 4096 token, che occupa circa 4 GB di VRAM; ridurlo a 512 porta l'occupazione a circa 0.1 GB, lasciando spazio per il modello RL. Il flag `keep_alive=-1` impedisce a Ollama di scaricare il modello dalla VRAM tra una chiamata e l'altra; senza questo flag il modello si scaricherebbe dopo pochi secondi di inattività e il warm-up si ripeterebbe ad ogni chiamata.

Se una chiamata va in timeout o la connessione si interrompe, il client riprova fino a tre volte, aspettando 0.5 secondi prima del primo tentativo, 1 secondo prima del secondo, 1.5 prima del terzo. Prima di ogni nuovo tentativo riapre la connessione HTTP, perché un errore di rete lascia spesso la connessione in uno stato inconsistente.

6.10 Problemi incontrati e soluzioni adottate

Reward sparsa. Con quattro casse su griglia 10×10 , l'esplorazione casuale non produce quasi mai una soluzione completa. Senza episodi di successo non c'è segnale per imparare. Il curriculum risolve il problema partendo da una sola cassa su livelli generati: in quella configurazione le soluzioni sono raggiungibili in poche decine di mosse, la reward di completamento arriva abbastanza spesso da avviare l'apprendimento, e da lì si può costruire gradualmente la competenza necessaria per le fasi successive.

CNN che impara la posizione invece della struttura. Nelle versioni v1–v7 del curriculum, le griglie di dimensione crescente venivano centrate in un frame 10×10 con padding. Ogni fase aveva un offset diverso. La CNN imparava a riconoscere la struttura del puzzle in base a dove si trovava nella griglia, non in base alla struttura in sé. Quando l'offset cambiava, il modello smetteva di funzionare. La soluzione è stata eliminare il padding: la griglia è rimasta fissa a 10×10 per tutto il training, variando solo il numero di casse.

LLM troppo lento per il training. Chiamare l'LLM ad ogni step avrebbe reso il training di AG-LLM-REW impraticabilmente lento. Tre accorgimenti applicati insieme hanno ridotto il problema a livelli accettabili: chiamare l'LLM solo quando una cassa viene effettivamente spinta (circa il 5% degli step), mettere in cache le valutazioni già calcolate, e usare la connessione HTTP persistente con contesto ridotto a 512 token. Il risultato è un training circa undici volte più lento di AG-PPO in tempo di calcolo, a parità di step: lento ma gestibile con un'esecuzione dedicata.

Qualità dell'LLM. Il primo modello testato, `llama3.1:8b`, produceva quasi esclusivamente wall-collision e risposte JSON malformate. Il fallback sull'azione casuale scattava così spesso che le dimostrazioni raccolte erano di fatto casuali. Il passaggio a `qwen3:14b-q4_K_M` ha risolto entrambi i problemi: zero fallback, nessuna wall-collision sistematica.

Formato delle osservazioni. `CnnPolicy` di SB3 richiede osservazioni `channels-first` (C, H, W) , ma `SokobanEnv` restituisce matrici (H, W) . Il wrapper `AggiuntaCanale`

aggiunge la dimensione del canale. La dimensione del flatten layer della CNN varia con la dimensione dell'input e viene calcolata automaticamente con una forward pass su un tensore dummy alla costruzione del modello, invece di un valore fisso nel codice.

Esperimenti e Risultati

7.1 Setup sperimentale

Tutti gli esperimenti usano seed 42. La valutazione è in modalità *best_per_fase*: per ogni agente RL si carica il checkpoint con performance migliore registrata durante il training (best model di `EvalCallback`), non il modello all'ultimo step. Questo evita di penalizzare agenti che hanno raggiunto il picco a metà training e poi regredito. Per AG-LLM-ACT non esiste checkpoint: l'LLM gira direttamente in inferenza.

Per ogni coppia (agente, fase) vengono eseguiti 100 episodi. Le metriche registrate sono:

- **Solve rate:** percentuale di episodi terminati con `terminated=True` (tutte le casse su target);
- **Reward cumulativa media:** somma delle reward per episodio, mediata su 100 episodi;
- **Mosse medie:** lunghezza media delle soluzioni trovate, calcolata solo sugli episodi con `terminated=True`;

- **Casse su target medie:** numero medio di casse correttamente posizionate a fine episodio.

I risultati completi sono in `results/seed42/risultati_comparativi_seed42.json` per il sistema 10×10 e in `results/seed42/risultati_7x7_seed42.json` per il sistema 7×7 .

7.2 Sistema 10×10 : risultati per fase

7.2.1 Fase C0: 1 cassa, livelli generati

Su C0 quattro agenti su cinque hanno solve rate non nullo: AG-PPO e AG-LLM-REW (entrambi 3%), AG-LLM-ACT (2%) e AG-LLM-GUIDE (1%). Solo AG-DQN resta a 0%.

I numeri sono bassi. Il modello caricato è il best checkpoint durante il training su distribuzioni variabili del curriculum: non è specializzato su C0 ma è il migliore trovato lungo l'intero percorso. I livelli C0 su griglia 10×10 hanno muri interni casuali e giocatore, cassa, target in posizioni casuali; la soluzione non è banale. Le reward cumulative sono tutte negative: -0.11 per PPO, -0.40 per LLM-ACT, -0.15 per LLM-REW. Nessun agente ha sviluppato una policy stabile nemmeno sul task più semplice del sistema 10×10 .

7.2.2 Fasi C1–C5: solve rate a zero

Dalla fase C1 (2 casse generate) in avanti, il solve rate è 0% per tutti gli agenti su tutti i 100 episodi valutati. Le reward peggiorano al crescere della difficoltà.

Fase	AG-PPO	AG-DQN	AG-LLM-ACT	AG-LLM-G.	AG-LLM-R.
C0	−0.110	−0.376	−0.404	−0.266	−0.150
C1	−0.592	−0.650	−0.878	−0.765	−0.565
C2	−0.744	−1.004	−1.124	−0.974	−0.863
C3	−0.967	−1.271	−1.259	−1.268	−1.206
C4	−1.051	−1.432	−1.190	−1.442	−1.181
C5	−1.186	−1.531	−1.375	−1.473	−1.368

AG-LLM-ACT sulle fasi C4 e C5 mostra un risultato interessante, infatti, seppure l’LLM non risolve nessun livello la sua metrica `casse_su_target` media è 0.21 su C4 e 0.19 su C5. Tutti gli agenti RL sul test indipendente *val-unfiltered* restano sotto 0.06. L’LLM posiziona più casse degli agenti RL pur non completando mai il puzzle: sa avvicinarsi ai target localmente, ma non tiene traccia delle dipendenze tra le quattro casse.

I test su *val-medium* e *val-unfiltered* (set di validazione indipendenti dal training) confermano il quadro: 0% per tutti, reward tra −0.99 e −1.53.

7.3 Sistema 7×7: risultati per fase

7.3.1 Fase C0: 1 cassa, livelli generati 7×7

Sul sistema 7 × 7 la fase C0 produce risoluzioni concrete per tutti e cinque gli agenti.

Agente	Solve rate	Reward	Mosse (risolti)
AG-PPO	16%	+1.619	22.88
AG-DQN	11%	+0.957	5.73
AG-LLM-ACT	9%	+0.563	25.44
AG-LLM-REW	9%	+0.704	33.67
AG-LLM-GUIDE	6%	+0.360	3.50

Tutti e cinque gli agenti hanno reward media positiva su C0: le soluzioni trovate compensano le penalità di step. AG-PPO è il migliore in assoluto, con solve rate 16% e reward media +1.62, davanti ad AG-DQN (11%, +0.96); AG-LLM-ACT e AG-LLM-REW seguono appaiati al 9%.

Le mosse medie raccontano due stili opposti. AG-DQN e AG-LLM-GUIDE risolvono in pochissime mosse (5.7 e 3.5, vicino all’ottimo per una sola cassa), mentre AG-PPO, AG-LLM-ACT e AG-LLM-REW raggiungono il bersaglio con percorsi molto più lunghi (rispettivamente 22.9, 25.4 e 33.7 mosse): trovano la soluzione ma vagano, segno di una pianificazione poco sistematica. AG-PPO, in particolare, risolve più livelli di tutti pur senza imboccare il percorso breve.

7.3.2 Fasi C1 e C2 con rispettivamente 2 e 3 casse

Da C1 il solve rate crolla quasi a zero: solo AG-PPO risolve un singolo livello su 100 (1%), tutti gli altri agenti restano a 0%. Le reward peggiorano rispetto a C0 ma restano meno negative del sistema 10×10 alla stessa complessità, per la griglia più piccola e lo spazio degli stati ridotto.

Fase	AG-PPO	AG-DQN	AG-LLM-ACT	AG-LLM-G.	AG-LLM-R.
C0 (1 cassa)	+1.619	+0.957	+0.563	+0.360	+0.704
C1 (2 casse)	−0.074	−0.504	−0.638	−0.470	−0.571
C2 (3 casse)	−0.522	−0.770	−0.848	−0.791	−0.578

7.4 Analisi comparativa

PPO vs DQN. Su C0 7×7 AG-PPO è il migliore in assoluto (16% contro l’11% di AG-DQN): la memoria ricorrente della LSTM, propagata lungo l’episodio, permette alla policy di portare a termine più livelli. AG-DQN resta competitivo e molto più efficiente nelle soluzioni (5.7 mosse contro le 22.9 di AG-PPO): il replay buffer off-policy riusa bene le poche transizioni vincenti trovate per esplorazione e converge su percorsi quasi ottimi. AG-PPO risolve di più, ma con traiettorie lunghe. Su C1 (2

casce) è il solo AG-PPO a superare lo 0% (1%): la memoria aiuta anche quando la difficoltà cresce, pur senza generalizzare.

LLM senza training. AG-LLM-ACT senza alcun addestramento RL raggiunge il 9% su C0 7×7 , pari ad AG-LLM-REW e poco sotto ad AG-DQN (11%), pur restando dietro ad AG-PPO (16%). Sul sistema 10×10 ottiene il 2% su C0. È un risultato notevole per un modello mai addestrato sul compito: l’LLM risolve alcuni livelli semplici per ragionamento locale. Su C1+ cade a 0%: due casce richiedono un lookahead che il modello con `think=False` non ha.

LLM-GUIDE. AG-LLM-GUIDE porta C0 7×7 al 6%, sotto sia AG-DQN (11%) sia AG-PPO (16%). Il pre-fill del replay buffer con dimostrazioni LLM dà un vantaggio sulle prime iterazioni, ma il DQN risultante non supera né l’AG-DQN addestrato con esperienza propria e curriculum, né l’AG-PPO ricorrente.

LLM-REW. AG-LLM-REW condivide l’architettura RecurrentPPO di AG-PPO: il confronto fra i due, a parità di rete e di budget, isola l’effetto netto della reward dell’LLM. Su C0 7×7 AG-LLM-REW raggiunge il 9% (pari ad AG-LLM-ACT), ma resta nettamente sotto ad AG-PPO (16%): la reward aggiuntiva non aiuta, anzi penalizza, oltre a rallentare il training di circa 11 volte. Il problema non è il wrapper ma la qualità del segnale: su livelli a più casce, giudicare localmente se una spinta ha avvicinato una cassa al target non cattura le dipendenze tra le casce e genera feedback contraddittori all’agente.

7.5 Discussione

I risultati del sistema 10×10 mostrano che il curriculum non ha portato all’apprendimento della task principale: 0% su C1+ per tutti gli agenti. Il problema non sembra essere di natura tecnica, in quanto l’ambiente funziona, i test passano, il curriculum avanza correttamente ma sembra essere legato al budget, infatti analizzando lo Stato dell’Arte, Guez et al. [3] usano 2 miliardi di step su hardware distribuito per raggiungere il

99% di solve rate. Questo progetto usa circa 9M di step per agente su singola GPU, circa 220 volte meno.

Il sistema 7×7 mostra che il curriculum funziona sul task più semplice: AG-PPO risolve il 16% dei livelli a una cassa, davanti ad AG-DQN (11%) e AG-LLM-ACT (9%). La quasi totale caduta su due casse (solo AG-PPO all'1%) mostra dove il budget non basta: la complessità combinatoria cresce abbastanza da rendere insufficiente anche il budget ridotto del sistema semplificato.

Tra gli agenti basati su LLM, il comportamento più informativo è quello di AG-LLM-ACT. Un LLM senza training RL raggiunge il 9% su C0 7×7 , vicino ad AG-DQN (11%), e supera tutti gli agenti RL sulla metrica di posizionamento delle casse su C4–C5. La capacità di ragionamento locale dell'LLM è reale per task semplici. Quella di lookahead su più casse no, almeno non con `think=False` e un contesto di 512 token.

8.1 Riepilogo dei contributi

Il progetto ha prodotto cinque agenti funzionanti, un ambiente Sokoban compatibile con Stable Baselines 3, un generatore di livelli con verifica di risolubilità (BFS esatto sui casi piccoli, euristica sui grandi) e 64 test unitari. Due configurazioni: griglia 10×10 con livelli Boxoban, griglia 7×7 con generazione procedurale.

Il curriculum adattivo a più fasi ha funzionato dove poteva funzionare: 1 cassa su 7×7 , con AG-PPO al 16%, AG-DQN all'11% e AG-LLM-ACT al 9% di solve rate. Su 4 casse su 10×10 il training non decolla e ciò è dovuto come abbiamo visto nel capitolo precedente al budget ridotto e all'hardware insufficiente.

8.2 Limiti

AG-LLM-REW è stato l'esperimento più costoso. Condividendo architettura e budget con AG-PPO, offre l'ablation più pulita: la reward locale dell'LLM non aiuta, anzi penalizza, e AG-PPO lo supera nettamente (16% contro 9% su C0 7×7). Il feedback locale (questa spinta ha avvicinato una cassa al target?) non vede l'interdipendenza tra le casse. Su puzzle a 4 casse produce segnali contraddittori e

rallenta il training di circa 11 volte senza migliorare i risultati. Con `think=True` la qualità del feedback sarebbe probabilmente migliore, ma la latenza passerebbe da 0.2 s a circa 2 s per chiamata: non praticabile su budget di training nell'ordine di milioni di step.

AG-LLM-GUIDE fallisce per un motivo diverso. Learning from Demonstrations funziona solo se il teacher sa fare la task. Su Boxoban a 4 casse, `qwen3:14b` risolve 0% degli episodi di raccolta. Le “dimostrazioni” sono traiettorie di fallimento; darle al DQN come pre-riempimento del replay buffer peggiora le cose rispetto alla sola esperienza propria.

8.3 Lettura dei risultati

Il dato più utile dell'intera valutazione è il comportamento di AG-LLM-ACT. Senza training RL raggiunge il 9% su C0 7×7 , vicino ad AG-DQN addestrato (11%) e dietro al solo AG-PPO ricorrente (16%). Su C4 Boxoban posiziona il 21% delle casse sui target, contro il 6% o meno degli agenti RL. Sa ragionare localmente sullo stato attuale. Non sa pianificare su più mosse, almeno non con `think=False` e un contesto di 512 token.

Il flag `think=False` era necessario per mantenere la latenza sotto 0.2 s durante la valutazione ed era la scelta giusta per rendere il training di AG-LLM-REW computazionalmente fattibile. È anche, probabilmente, la scelta che ha tolto all'LLM l'unico meccanismo che potrebbe migliorarne le prestazioni su puzzle complessi: la catena di pensiero interna.

8.4 Sviluppi possibili

I risultati presentati si basano su un'unica esecuzione con seed 42. Le sementi 123 e 456 erano pianificate fin dall'inizio del progetto ma non sono state completate per vincoli di tempo. Senza una validazione multi-seed non è possibile distinguere le performance stabili da quelle dovute alla casualità dell'inizializzazione: il 16% di AG-PPO su C0 7×7 potrebbe essere un risultato robusto o un caso fortunato. Rieseguire entrambi i sistemi sui tre seed è la priorità più immediata, perché non richiede

modifiche al codice e trasforma i risultati attuali da indicativi a statisticamente fondati.

La seconda direzione riguarda l’infrastruttura di calcolo. Il sistema di riferimento su Boxoban, IMPALA+DRC di Guez et al., è stato addestrato per circa 2 miliardi di step su hardware distribuito. Il budget di questo progetto è di circa 9 milioni di step su una singola RTX 4070: un rapporto di oltre 200 a 1. Non si può concludere che il curriculum adattivo non funziona su Boxoban; si può concludere che non funziona con questo budget. Accedere a infrastrutture di calcolo distribuito, o semplicemente a una sessione multi-GPU più lunga, permetterebbe di esplorare le fasi C3–C5 con budget paragonabili a quelli della letteratura.

La terza direzione nasce direttamente da una scelta tecnica di questo progetto. Il flag `think=False` è stato necessario per mantenere la latenza sotto 0.2 s per chiamata durante il training di AG-LLM-REW; con `think=True` la latenza sale a circa 2 s. Non è però noto quanto valga la catena di ragionamento interno di qwen3 sulla qualità delle decisioni. Un esperimento limitato di 100 episodi su C0 7×7 con `think=True` misurerebbe questa differenza senza costi proibitivi. Se il gap fosse significativo, per esempio dal 9% al 25% di solve rate, cambierebbe la lettura dei risultati di AG-LLM-ACT e AG-LLM-GUIDE: il problema sarebbe la latenza, non il ragionamento.

Infine, tutti gli esperimenti LLM di questo progetto usano `qwen3:14b-q4_K_M`, scelto perché gira localmente senza costi. Modelli significativamente più capaci, come GPT-4o o Claude 3.5, hanno mostrato in letteratura capacità di pianificazione su puzzle logici molto superiori. Testarli su AG-LLM-ACT e AG-LLM-GUIDE, anche solo su un campione di livelli C0–C1, permetterebbe di rispondere a una domanda rimasta aperta: il fallimento dei due agenti basati su LLM dipende dal modello specifico scelto o dalla struttura del problema? Se un modello top-tier raggiungesse solve rate significativi su C1, le architetture AG-LLM-ACT e AG-LLM-GUIDE avrebbero senso da sviluppare ulteriormente con accesso API.

Bibliografia

- [1] J. C. Culberson, “Sokoban is pspace-complete,” Department of Computing Science, University of Alberta, Technical Report TR97-02, 1997. (Citato alle pagine 1 e 16)
- [2] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. d. Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. J. S. Tan, and O. G. Younis, “Gymnasium,” 2023. [Online]. Available: <https://zenodo.org/record/8127026> (Citato a pagina 2)
- [3] A. Guez, M. Mirza, K. Gregor, Q. Huys, T. Lattimore, T. Weber, Y. Zhu, S. Osindero, K. Kavukcuoglu, and D. Hassabis, “An investigation of model-free planning,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR, 2019, dataset Boxoban disponibile su: <https://github.com/google-deepmind/boxoban-levels>. (Citato alle pagine 2, 13, 16 e 43)
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. (Citato a pagina 5)
- [5] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992. (Citato a pagina 7)
- [6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie,

- A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015. (Citato alle pagine 7 e 12)
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. (Citato a pagina 7)
- [8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. (Citato a pagina 8)
- [9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. (Citato a pagina 9)
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. (Citato a pagina 9)
- [11] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*. ACM, 2009, pp. 41–48. (Citato alle pagine 10 e 13)
- [12] T. Hester, M. Vecerík, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, I. Osband, J. Agapiou, A. Gruslys, and J. Z. Leibo, "Deep Q-learning from demonstrations," in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*. AAAI Press, 2018. (Citato alle pagine 11 e 14)
- [13] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016. (Citato a pagina 12)
- [14] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre,

- G. van den Driessche, T. Graepel, and D. Hassabis, “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018. (Citato a pagina 12)
- [15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. (Citato a pagina 14)
- [16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023. (Citato a pagina 14)
- [17] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askeel, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. (Citato a pagina 14)